National University of Computer and Emerging Sciences

FAST School of Computing, Karachi Campus

Spring 2026

Software Requirements Specification
Electrolux Energy Management System (EMS)

Course:	CS3009 – Software Engineering
Instructor:	Asst. Prof. Engr. Abdul Rahman
Standard:	IEEE Std 830-1998
Version:	1.0 – Final
Date:	May 2026

Prepared By:

Name	Roll No.	Role
Huzaifa Abdul Rehman	23K-0782	Full-stack Developer
Muhammad Abdullah Khan	23K-0607	Full-stack Developer
Abdul Moiz Hussain	23K-0553	Full-stack Developer

Prepared in conformance with IEEE Std 830-1998:
Recommended Practice for Software Requirements Specifications

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Project Overview and Abstract . . . 3
1.3 Problem Statement . . . 3
1.4 Project Vision . . . 3
1.5 Scope . . . 3
1.6 Definitions and Acronyms . . . 3
1.7 References . . . 4

2 Overall Description **4**
2.1 Product Perspective . . . 4
2.2 User Classes . . . 4
2.3 Operating Environment . . . 4
2.4 Constraints . . . 4
2.5 Assumptions . . . 5
2.6 Feasibility Study . . . 5

3 Functional Requirements **5**
3.1 Authentication and Access Control . . . 5
3.2 Billing and Tariff Management . . . 5
3.3 Payment Processing . . . 6
3.4 Meter Reading Management . . . 6
3.5 Complaint Management . . . 6
3.6 Work Order Management . . . 6
3.7 Outage Management . . . 7
3.8 New Connection Application . . . 7
3.9 Customer and Employee Management (Admin) . . . 7
3.10 Analytics, Notifications, and Data Export . . . 7

4 Non-Functional Requirements **8**

5 System Architecture and Design **8**
5.1 Technology Stack . . . 8
5.2 Database Schema (Key Tables) . . . 9
5.3 Key Design Optimizations . . . 9
5.4 REST API Summary . . . 9

6 Use Case Model **9**
6.1 Actors and Use Case Diagram . . . 10
6.2 Use Case Summary . . . 10
6.3 Detailed Use Case: UC-03 – Pay Bill . . . 12
6.4 Detailed Use Case: UC-12 – Generate Bill . . . 12

7 Development Methodology and Timeline **12**
7.1 Agile Scrum . . . 12
7.2 Tools and Infrastructure . . . 13

8 Existing and Related Work **13**

9 Appendix **13**
9.1 20 Core Features . . . 13
9.2 Running the Project . . . 13
9.3 Repository . . . 14

9.4 Environment Configuration 14
9.5 Demo Credentials (after seeding) 14
9.6 Database Tables (19) 14

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **Electrolux Energy Management System (EMS)**, a web-based electricity utility management platform. It is intended for the development team, the course instructor, and any future stakeholders.

1.2 Project Overview and Abstract

The Electrolux EMS is a comprehensive web-based solution designed to modernise electricity utility operations in Karachi, Pakistan. Addressing the operational gaps in K-Electric's existing digital services, the system provides a centralised digital platform for three user roles: **Customers** (consumers), **Employees** (field staff), and **Administrators** (management). Customers gain transparent access to bills, payment tracking, usage analytics, and complaint management. Employees digitally record meter readings and manage work orders. Administrators oversee operations through dashboards, bulk billing, tariff management, and outage coordination. Built using Next.js, TypeScript, MySQL, and Drizzle ORM with role-based authentication and automated billing calculations, the system bridges the gap between utility providers and consumers.

1.3 Problem Statement

Despite K-Electric offering basic digital services (online bill viewing via KE Live app, e-billing by email, and complaint registration via app/WhatsApp), significant gaps remain. Customers lack usage analytics, bill estimation tools, and transparent complaint lifecycle tracking. Meter reading requests cannot be submitted digitally. Employees have no integrated work order or meter recording platform. Administrators lack bulk billing, tariff slab configuration, and zone-based outage management tools in a single system. These operational gaps fragment workflows, cause delays, and limit accountability across the utility management cycle.

1.4 Project Vision

To build a transparent, efficient, and accessible digital utility management platform that empowers electricity consumers in Karachi while enabling K-Electric staff and administrators to manage operations with greater accuracy and accountability — mirroring the standards of modern digital utility services globally, tailored to Karachi's constraints.

1.5 Scope

In Scope: Web-based portals for Customer, Employee, and Administrator roles; automated tiered tariff bill generation; meter reading management; payment tracking; complaint lifecycle management; outage management; new connection applications; analytics and reporting; 20 core features.

Out of Scope: Native mobile apps, real payment gateway integration, IoT/smart-meter integration, SMS/email notifications, production deployment.

1.6 Definitions and Acronyms

- **EMS** – Energy Management System
- **SRS** – Software Requirements Specification
- **RBAC** – Role-Based Access Control
- **JWT** – JSON Web Token
- **ORM** – Object-Relational Mapping
- **ACID** – Atomicity, Consistency, Isolation, Durability
- **kWh** – Kilowatt-hour
- **FR / NFR** – Functional / Non-Functional Requirement
- **NEPRA** – National Electric Power Regulatory Authority
- **GST** – Goods and Services Tax

1.7 References

[1] IEEE Std 830-1998, *IEEE Recommended Practice for SRS*. IEEE, 1998.
[2] K-Electric, "Customer Services." https://www.ke.com.pk. [Feb. 2026].
[3] V. Verma & A. Sharma, "Electricity Billing Management System," *IJCA*, vol. 120, no. 21, 2015.
[4] Next.js Docs. https://nextjs.org/docs. [Feb. 2026].
[5] Drizzle ORM Docs. https://orm.drizzle.team. [Feb. 2026].
[6] A. Stellman & J. Greene, *Learning Agile*. O'Reilly, 2014.
[7] NEPRA, https://www.nepra.org.pk. [Feb. 2026].
[8] T. Connolly & C. Begg, *Database Systems*, 6th ed. Pearson, 2015.

2. Overall Description

2.1 Product Perspective

Electrolux EMS is a new, self-contained web application with no dependency on existing proprietary utility systems. It follows a classic **three-tier architecture**:

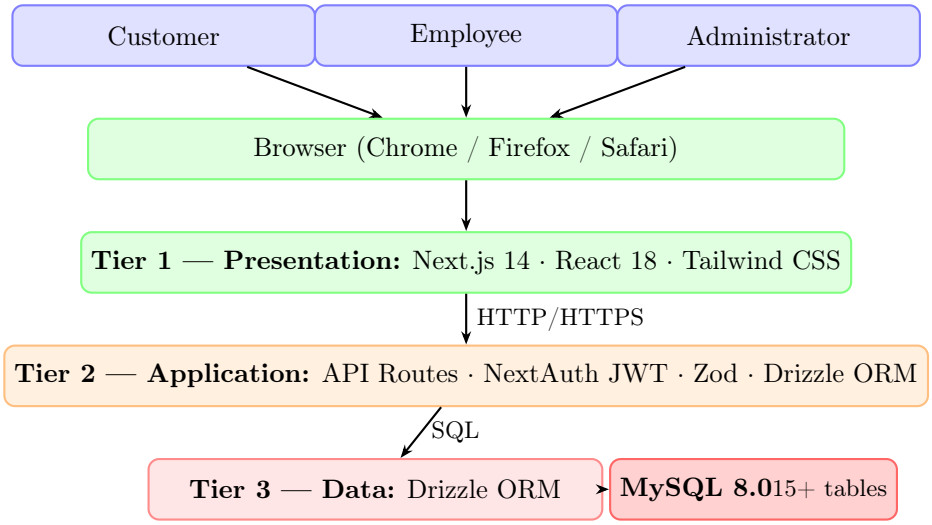

Figure 1: Three-Tier System Architecture

2.2 User Classes

Role	Description	Key Responsibilities
Customer	Registered electricity consumer	View bills, pay, track complaints, request readings
Employee	Field/office utility staff	Record meter readings, manage work orders
Administrator	Utility management	Full system oversight, billing, tariffs, analytics
Public User	Unauthenticated visitor	Submit and track new connection applications

2.3 Operating Environment

Node.js 18+, MySQL 8.0, any modern browser (Chrome 90+, Firefox 88+, Safari 14+). Cross-platform server (Windows, macOS, Linux). HTTP on port 3000 (dev) / HTTPS 443 (prod).

2.4 Constraints

- Stack fixed: Next.js 14, TypeScript, MySQL 8.0, Drizzle ORM, NextAuth.js v4.

- All passwords hashed via `bcryptjs` (≥10 salt rounds); no plaintext storage.
- No real payment gateway; all payments are simulated/recorded only.
- Development constrained to 8 weeks (Spring 2026 semester).

2.5 Assumptions

MySQL 8.0 available locally for demonstration; `.env` configured before launch; active tariff data seeded before bill generation; evaluation on `localhost:3000`.

2.6 Feasibility Study

Technical	All technologies are open-source and well-documented. Team has full-stack skills in TypeScript, React, Next.js, and MySQL. No proprietary licences required.
Operational	Web-based; users need only a browser. Role-based UI mirrors familiar workflows, minimising learning curve. Admin dashboard replaces manual reporting.
Economic	Zero licensing costs (open-source stack). Student labour at no cost. Real-world savings from eliminated paper billing and reduced manual operations far outweigh development cost.

3. Functional Requirements

The 20 core features are grouped by domain. Each requirement uses **FR-XX** identifiers. *Shall* = mandatory.

3.1 Authentication and Access Control

FR-01	The system shall authenticate users via email and password; issue a JWT session token valid for 24 hours.
FR-02	All passwords shall be hashed using `bcryptjs` (≥10 salt rounds) before storage.
FR-03	The system shall enforce route-level RBAC: `/customer`, `/employee`, `/admin` paths restricted to respective roles.
FR-04	The system shall force a password change on first login (`requiresPasswordChange` flag).
FR-05	Customers shall submit password reset requests; administrators shall process them with token expiry tracking.

3.2 Billing and Tariff Management

Tiered Tariff Slab Calculation Algorithm

Let U = units consumed, slabs $\{s_i\}$ ordered by `slabOrder` with range $[a_i, b_i)$ and rate r_i:

$$B = \sum_{i=1}^{n} \min\bigl(\max(U - a_i,\ 0),\ b_i - a_i\bigr) \times r_i$$

Total bill = B + fixed charge + electricity duty + GST. This ensures progressive pricing where lower-consumption households pay proportionally less.

FR-06	The system shall generate monthly bills using the progressive slab algorithm above; store `tariffId` for audit.
FR-07	Each bill shall carry a unique number (`BILL-YYYY-XXXXXX`), default due date (+15 days), and status (generated / issued / paid / overdue / cancelled).
FR-08	The system shall reject duplicate bills: no two bills for the same customer and billing month.
FR-09	Administrators shall generate bills for all active customers in one bulk operation; the system shall report generated / skipped / failed counts.
FR-10	Customers shall view bills with full charge breakdown (base, fixed, duty, GST, total) and download/print them.
FR-11	Customers shall use a bill estimation calculator: enter estimated units, receive projected cost breakdown against the current tariff.
FR-12	Administrators shall define tariffs per connection type (Residential / Commercial / Industrial / Agricultural) with fixed charge, duty%, GST%, effective date, and unlimited progressive slabs.

3.3 Payment Processing

FR-13	The system shall record payments via: credit card, debit card, bank transfer, cash, cheque, UPI, wallet.
FR-14	Payment recording shall execute as an ACID transaction: atomically insert payment, update bill status, and decrement customer outstanding balance; all three succeed or all roll back.
FR-15	A bill is marked `paid` only when payment amount $\geq$ bill total (tolerance PKR 0.01). Each payment gets a unique receipt number (`RCP-YYYY-XXXXXX`) and transaction ID.

3.4 Meter Reading Management

FR-16	Employees shall record current and previous meter readings (kWh); system computes units consumed = current − previous (negative values accepted for meter rollover/replacement).
FR-17	Employees shall record meter condition (good / fair / poor / damaged) and accessibility (accessible / partial / inaccessible) with optional notes.
FR-18	Customers shall submit meter reading requests; administrators assign them to employees, auto-generating a `meter_reading` work order.

3.5 Complaint Management

FR-19	Customers shall submit complaints with category (power outage / billing / service / meter issue / connection / other), title, and description.
FR-20	Complaint lifecycle: submitted → under_review → assigned → in_progress → resolved → closed.
FR-21	Administrators shall assign priority (low / medium / high / urgent) and assign to an employee. The complaint queue shall be sorted: urgent first, then FIFO within each priority.

3.6 Work Order Management

FR-22	Employees shall view assigned work orders (types: meter_reading, maintenance, complaint_resolution, new_connection, disconnection, reconnection), update status (assigned → in_progress → completed), and record completion notes.

3.7 Outage Management

FR-23	Administrators shall create outage records: area, zone, type (planned/unplanned), severity (low/medium/high/critical), scheduled times. Status: scheduled → ongoing → restored / cancelled.
FR-24	Customers shall view the outage schedule for their assigned zone from the customer portal.

3.8 New Connection Application

FR-25	Any unauthenticated visitor shall submit a new connection application (name, ID proof, contact, property details, load required). Each application gets a unique `APP-YYYY-XXXXXX` number, trackable without login.
FR-26	The system shall reject duplicate applications (same email or phone in an active application). Status: pending → under_review → approved → connected (or rejected). On approval, account number and temporary password are auto-generated.

3.9 Customer and Employee Management (Admin)

FR-27	Administrators shall perform full CRUD on customer accounts. Each customer has a unique account number (`ELX-YYYY-XXXXXX`) and meter number (`MTR-XXX-XXXXXX`). Status: pending_installation / active / suspended / inactive.
FR-28	Administrators shall perform full CRUD on employee accounts. Each employee has a unique employee number, designation, department, assigned zone, and status (active / inactive).

3.10 Analytics, Notifications, and Data Export

FR-29	Customer analytics: monthly consumption trends and cost patterns (Chart.js bar/line charts).
FR-30	Admin analytics: top 10 revenue customers, monthly collection trends, consumption by month, revenue by connection type, payment method distribution, work order status breakdown.
FR-31	Administrators shall export system data (customers, bills, payments, meter readings, work orders, employees, tariffs) in CSV or JSON format.
FR-32	The system shall generate notifications for: bill issuance, payment confirmation, work order assignment, complaint status changes, outage schedules. Notification failures shall not block primary operations.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API endpoints shall respond within 2 seconds under normal single-user development load.
NFR-02	Performance	All DB joins and filters shall use indexed columns; list endpoints shall support pagination (default 10/page).
NFR-03	Security	Passwords hashed via bcryptjs (≥10 rounds); never stored as plaintext.
NFR-04	Security	Every API route validates session and user role before processing; cross-customer data access blocked.
NFR-05	Security	Admin-only endpoints return HTTP 403 for non-admin sessions. All queries use parameterised statements (Drizzle ORM) preventing SQL injection.
NFR-06	Security	JWT secrets and DB credentials stored in `.env` only; never committed to repository.
NFR-07	Reliability	Multi-table financial operations (payments) use ACID transactions; partial write failures roll back completely.
NFR-08	Usability	Fully responsive UI (Tailwind CSS) on desktop (≥1280px) and mobile (≥320px). Inline form validation errors. Role-specific landing dashboard after login.
NFR-09	Maintainability	TypeScript strict mode throughout. DB schema via Drizzle migrations only. ESLint passes with zero errors at build time.
NFR-10	Scalability	Denormalised running totals (`outstandingBalance`, `averageMonthlyUsage`) in customer table for O(1) dashboard reads.

5. System Architecture and Design

5.1 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14, React 18	App Router, SSR, page routing
Language	TypeScript 5.3	End-to-end type safety (strict mode)
Styling	Tailwind CSS 3.4	Responsive utility-first UI
Animation	Framer Motion, Three.js/Vanta	Page transitions, 3D animated home page
Charts	Chart.js, react-chartjs-2	Consumption and analytics visualisations
Forms	React Hook Form, Zod	Form management and runtime validation
Auth	NextAuth.js v4	JWT sessions, credentials provider
Security	bcryptjs	Password hashing
ORM	Drizzle ORM 0.44.7	Type-safe queries, schema migrations
Database	MySQL 8.0	Relational persistence and integrity
HTTP	Axios 1.6	Client-side API requests

5.2 Database Schema (Key Tables)

Table	Key Fields
`users`	id, email (UNIQUE), password (hashed), userType (admin/employee/customer), isActive, requiresPasswordChange
`customers`	id, userId (FK), accountNumber (UNIQUE), meterNumber (UNIQUE), connectionType, zone, status, outstandingBalance, averageMonthlyUsage
`employees`	id, userId (FK), employeeNumber (UNIQUE), designation, department, assignedZone, status
`bills`	id, billNumber (UNIQUE), customerId (FK), billingMonth, unitsConsumed, baseAmount, fixedCharges, electricityDuty, gstAmount, totalAmount, status, tariffId (FK)
`meterReadings`	id, customerId (FK), currentReading, previousReading, unitsConsumed, meterCondition, accessibility, employeeId (FK)
`payments`	id, customerId (FK), billId (FK), paymentAmount, paymentMethod, transactionId (UNIQUE), receiptNumber (UNIQUE), status
`tariffs`	id, category, fixedCharge, electricityDutyPercent, gstPercent, effectiveDate
`tariffSlabs`	id, tariffId (FK, CASCADE), slabOrder, startUnits, endUnits, ratePerUnit
`complaints`	id, customerId (FK), employeeId (FK), category, status, priority, submittedAt, resolvedAt, resolutionNotes
`workOrders`	id, employeeId (FK), customerId (FK), workType, status, priority, assignedDate, dueDate
`connectionRequests`	id, applicationNumber (UNIQUE), applicantName, email, connectionType, status, accountNumber (on approval)
`outages`	id, areaName, zone, outageType, severity, scheduledStartTime, scheduledEndTime, status
`notifications`	id, userId (FK), notificationType, title, message, isRead, priority, actionUrl

Key relationships: users ↔ customers (1:1), users ↔ employees (1:1), customers → bills/payments/complaints/meterReadings (1:N), tariffs → tariffSlabs (1:N, cascade delete), employees → workOrders (1:N).

5.3 Key Design Optimizations

Technique	Description
Progressive Tariff Slabs	Sequential slab algorithm applies lower rates to lower consumption bands, matching NEPRA tariff structure.
ACID Payment Transaction	Insert payment + update bill + update customer balance in one atomic DB transaction; all-or-nothing.
Priority Complaint Queue	Sorted urgent → high → medium → low, then FIFO within each level.
Denormalised Totals	`outstandingBalance` and `averageMonthlyUsage` updated at write-time for O(1) dashboard reads.
Parameterised Queries	Drizzle ORM `$dynamic()` composes conditional WHERE clauses with parameterised inputs; prevents SQL injection.

5.4 REST API Summary

The system exposes ≈97 RESTful endpoints under `/api/` returning `application/json`. Resources: `auth`, `customers`, `bills`, `payments`, `meter-readings`, `complaints`, `work-orders`, `connection-requests`, `tariffs`, `outages`, `employees`, `analytics`, `notifications`, `admin/data-export`. All endpoints validate session and role before processing.

6. Use Case Model

6.1 Actors and Use Case Diagram

Actors: **Customer** – authenticated electricity consumer. **Employee** – field/office staff. **Administrator** – utility management. **Public User** – unauthenticated (connection application only).

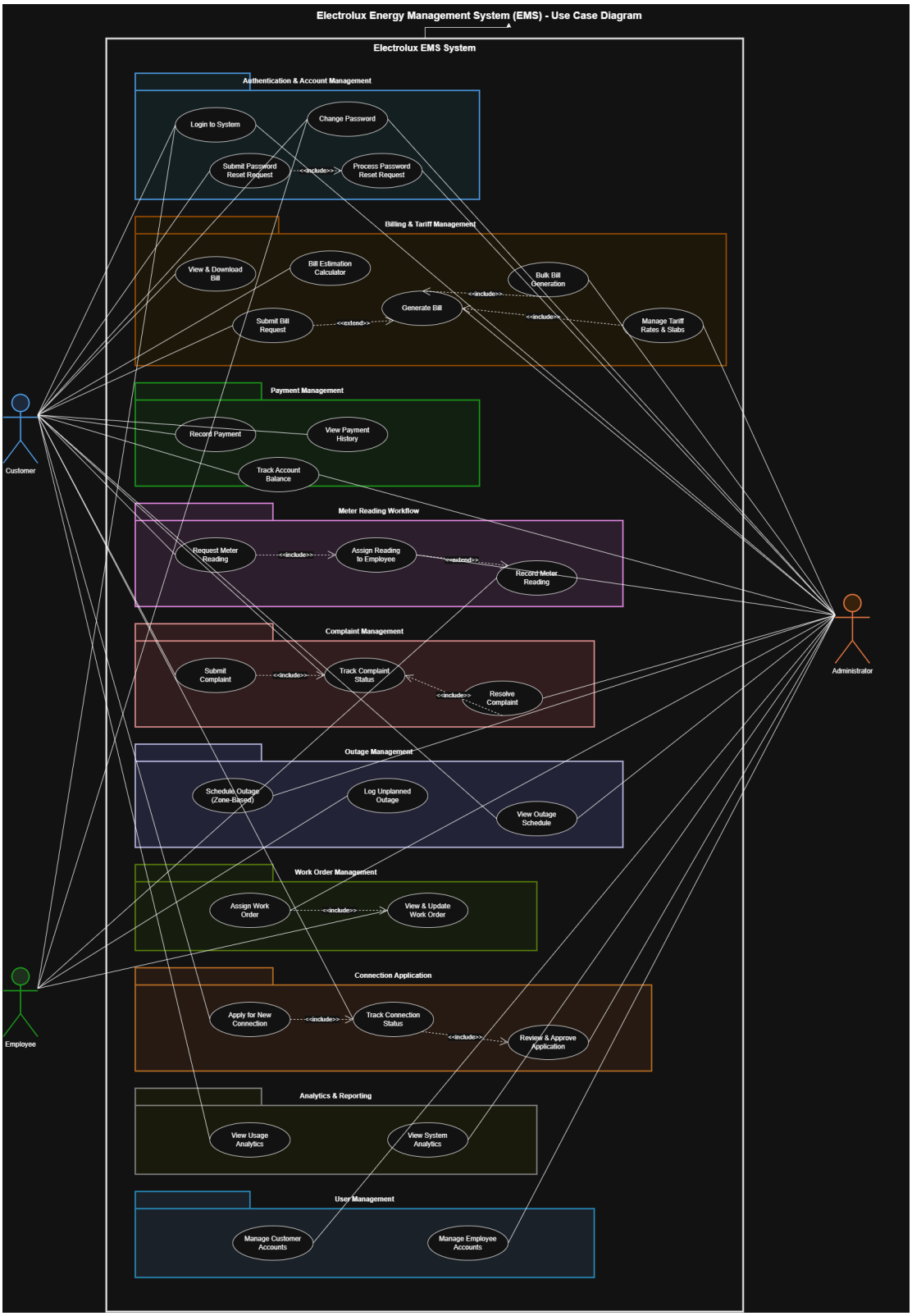

Figure 2: Use Case Diagram – Electrolux Energy Management System

6.2 Use Case Summary

Table 1: System Use Cases

Use Case	**Actor(s)**	**Description**
UC-01: Login / Logout	All	Authenticate; receive JWT session; invalidate on logout.
UC-02: View & Download Bills	Customer	View bills with full charge breakdown; download/print.
UC-03: Pay Bill	Customer	Record payment atomically; update bill and balance.
UC-04: Estimate Bill	Customer	Enter units; receive projected cost via tariff slabs.
UC-05: View Usage Analytics	Customer	Monthly consumption trends and cost charts.
UC-06: Submit Complaint	Customer	Submit complaint; initiate status lifecycle.
UC-07: Request Meter Reading	Customer	Submit reading request through the portal.
UC-08: Apply for Connection	Public User	Submit new connection application without login.
UC-09: Track Application	Public User	Track status by application number or email.
UC-10: Record Meter Reading	Employee	Record current/previous readings for a customer.
UC-11: Manage Work Orders	Employee	View, update, and complete assigned work orders.
UC-12: Generate Bill	Admin, Employee	Generate bill using meter reading and active tariff.
UC-13: Bulk Bill Generation	Admin	Generate bills for all active customers at once.
UC-14: Manage Customers	Admin	Full CRUD on customer accounts.
UC-15: Manage Employees	Admin	Full CRUD on employee accounts.
UC-16: Configure Tariffs	Admin	Define tariff categories and progressive slab rates.
UC-17: Assign Complaint	Admin	Prioritise and assign complaint to an employee.
UC-18: Schedule Outage	Admin	Create and manage zone-based power outage records.
UC-19: Process Connection	Admin	Approve or reject new connection applications.
UC-20: View Analytics	Admin	Revenue, consumption, complaint, payment analytics.
UC-21: Export Data	Admin	Export system data to CSV or JSON.
UC-22: Assign Reading Request	Admin	Assign customer reading request to an employee.
UC-23: Process Password Reset	Admin	Review and process customer password reset requests.

6.3 Detailed Use Case: UC-03 – Pay Bill

Primary Actor	Customer
Preconditions	Customer authenticated; at least one bill with status `issued` or `overdue`.
Postconditions	Payment inserted; bill status updated; customer balance decremented. All atomic.
Main Flow	1. Customer selects an outstanding bill and clicks "Pay Now". 2. Customer selects payment method and enters amount. 3. System opens a DB transaction; inserts payment record with unique receipt number. 4. If amount $\geq$ bill total, bill marked `paid`; customer balance decremented. 5. Transaction commits; confirmation notification dispatched; receipt shown.
Alt / Exception	DB failure $\Rightarrow$ full rollback, no partial state. Zero/negative amount $\Rightarrow$ HTTP 400.

6.4 Detailed Use Case: UC-12 – Generate Bill

Primary Actor	Administrator / Employee
Preconditions	Meter reading exists for the billing period; active tariff configured for customer's connection type.
Postconditions	New bill record created with correct charge breakdown and unique bill number.
Main Flow	1. Admin selects customer and billing month. 2. System fetches meter reading; computes units consumed. 3. System retrieves active tariff slabs; applies progressive algorithm. 4. System computes: base + fixed charge + duty + GST = total. 5. Bill record created (`BILL-YYYY-XXXXXX`), due date = issue date + 15 days. 6. Customer notified; `lastBillAmount` updated.
Alt / Exception	Duplicate bill $\Rightarrow$ error. No reading $\Rightarrow$ HTTP 400. No tariff $\Rightarrow$ HTTP 400.

7. Development Methodology and Timeline

7.1 Agile Scrum

Electrolux EMS was developed using **Agile Scrum** with four 2-week sprints over 8 weeks. All three team members participated equally across frontend, backend, database, and testing.

Sprint	Weeks	Deliverables
Sprint 1	1–2	Project setup, GitHub, MySQL schema (15+ tables), NextAuth authentication, role-based portal structure.
Sprint 2	3–4	Bill viewing & download, bill calculator, payment tracking, usage analytics, billing engine (tiered tariff), bulk bill generation.
Sprint 3	5–6	Meter reading recording, reading requests & assignment, work orders, customer/employee management, tariff management, connection application.
Sprint 4	7–8	Complaint tracking, outage management, bill request workflow, password reset, account balance, system testing, documentation.

7.2 Tools and Infrastructure

Languages: TypeScript, JavaScript, SQL
Framework: Next.js 14, React 18
Database: MySQL 8.0
ORM: Drizzle ORM + Drizzle Kit
Auth: NextAuth.js v4, bcryptjs, JWT
UI: Tailwind CSS, Framer Motion, Lucide React
Charts: Chart.js, react-chartjs-2
Dev Tools: VS Code, Git/GitHub, Postman
DB Tools: MySQL Workbench, Drizzle Studio
Diagrams: draw.io, ProjectLibre

8. Existing and Related Work

Feature	K-Electric	SmartGrid (US)	Electrolux EMS
Online Bill Viewing & PDF Download	Yes	Yes	**Yes**
Online Complaint Registration	Yes	No	**Yes**
Complaint Lifecycle Tracking	Basic	No	**Yes**
Bill Estimation Calculator	No	No	**Yes**
Usage Analytics / Graphs	No	Yes	**Yes**
Zone-Based Outage Management	No	No	**Yes**
Meter Reading Request (Customer)	No	No	**Yes**
New Connection Application	Partial	No	**Yes**
Employee Work Order Tools	No	Yes	**Yes**
Admin Bulk Bill Generation	No	No	**Yes**
Tiered Tariff Slab Calc.	No	No	**Yes**
Local Pakistan / NEPRA Focus	Yes	No	**Yes**

While K-Electric provides basic digital services (online bills, complaint registration, e-billing), Electrolux EMS is the only solution providing a fully integrated role-based platform covering internal operations (bulk billing, tariff management, work orders, employee tools) alongside customer-facing analytics, bill estimation, and zone-based outage management — all tailored to NEPRA tariff regulations and Karachi's infrastructure.

9. Appendix

9.1 20 Core Features

1. Role-Based Portal System
2. Electricity Bill Generation Engine
3. Tiered Tariff Slab Calculation
4. Bulk Bill Generation with Preview
5. Online Bill Viewing & Download
6. Bill Estimation Calculator
7. Payment Recording & Tracking
8. Meter Reading Recording (Employee)
9. Meter Reading Request (Customer)
10. Reading Request Assignment (Admin)
11. Employee Work Order Management
12. Customer Management (Admin)
13. Employee Management (Admin)
14. Tariff Rate Management
15. Customer Usage Analytics
16. Admin Analytics & Reporting
17. Complaint Submission & Tracking
18. Zone-Based Outage Management
19. New Connection Application & Tracking
20. Password Reset Request & Tracking

9.2 Running the Project

```
npm install                     # Install dependencies
cp .env.example .env            # Configure DB credentials
npx drizzle-kit push            # Push schema to MySQL
npx tsx src/scripts/seed.ts     # Seed demo data (optional)
npm run dev                     # Start dev server -> localhost:3000
```

9.3 Repository

https://github.com/HuzaifaAbdulRehman/electrolux_ms

9.4 Environment Configuration

Create a `.env.local` file in the project root before running:

```
# Database
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=electricity_ems

# NextAuth
NEXTAUTH_SECRET=your_random_secret_string
NEXTAUTH_URL=http://localhost:3000
```

9.5 Demo Credentials (after seeding)

Role	Email	Password
Administrator	admin@electrolux.com	admin123
Employee	employee@electrolux.com	employee123
Customer	customer@electrolux.com	customer123

All passwords are hashed using `bcryptjs` ($\geq$10 salt rounds) before storage. Plaintext passwords above are only valid immediately after the seed script runs.

9.6 Database Tables (19)

- `users`
- `customers`
- `employees`
- `bills`
- `payments`
- `tariffs`
- `tariffSlabs`
- `meterReadings`
- `complaints`
- `workOrders`
- `connectionRequests`
- `connectionApplications`
- `outages`
- `notifications`
- `readingRequests`
- `billRequests`
- `passwordResetRequests`
- `systemSettings`